第 2 章：基本数据类型

本章目标：用 Java 类型体系作参照，理解 Python 内置标量对象。配套代码：`example.py / Chapter02BasicTypes.java` 。

学习目标

学完本章后，你应该能够：

- 区分 `int` 、 `float` 、 `bool` 、 `str` 和 `None` ，并正确判断对象类型；
- 解释 Python 整数、除法和 Java 的关键语义差异；
- 识别浮点精度问题，并为金额选择更合适的数据表示；
- 理解真假值以及 `and` 、 `or` 返回操作数的规则；
- 正确处理缺失值、字符串转换和多行文本。

2.1 Python 的值都是对象

Python 没有 Java 的 primitive / wrapper 二分法。 `42` 、 `True` 和 `"Python"` 都是对象，
变量名只是绑定到对象。对象携带自己的类型，因此同一个名字以后可以重新绑定到另一种类型；
类型提示能帮助 IDE 和静态检查器，但默认不会在运行时强制类型。

常用标量类型如下：

Python	常见 Java 对照	关键差异
`int`	`int` / `long` / `BigInteger`	精度按需增长，受可用内存限制
`float`	`double`	通常是 IEEE 754 双精度浮点数
`bool`	`boolean`	字面量是 `True` / `False` ，也是 `int` 的子类
`str`	`String`	Unicode 文本且不可变；没有单独的 `char` 类型
`None`	`null`	表示"没有值"的单例对象，类型是 `NoneType`

查看类型： `type` 与 `isinstance`

README × README ×
'lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-e1f9e52c0bfb4615c10618a113.html
... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

查看类型：`type` 与 `isinstance`

`type(value)` 返回对象的准确类型；`isinstance(value, SomeType)` 还会考虑继承关系，通常更适合业务判断：

```
value = 42
print(type(value).__name__)    # int
print(isinstance(value, int))  # True
```

一个容易意外的边界是：`bool` 是 `int` 的子类，所以 `isinstance(True, int)` 为 `True`。如果业务要求“必须是整数，但不能是布尔值”，可写 `type(value) is int`。平时不要利用 `True == 1` 做业务计算，它会让代码含义变模糊。

2.2 类型转换不是类型强制

`int`、`float`、`str` 和 `bool` 既是类型，也能像函数一样构造新对象：

```
count = int("42")      # 42
ratio = float("0.75")  # 0.75
label = str(42)        # "42"
```

转换只接受符合规则的输入。`int("3.14")` 不会先转为浮点数再舍去小数，而会抛出 `ValueError`。若输入本来是小数字符串，必须先明确业务需要的是四舍五入、向下取整，还是拒绝输入，不能靠隐式猜测。

还要特别注意 `bool` 对字符串判断的是“是否为空”，并不解析英文含义：

```
bool("")       # False
bool("False")  # True，因为它是非空字符串
```

解析配置项时应该显式定义允许值，例如把去除空白并转为小写后的 `"true"`、`"1"` 视为真，而不是直接调用 `bool(text)`。

2.3 整数：精度、字面量与除法

Python 的 `int` 不区分 `byte`、`short`、`int` 和 `long`。结果超出 32 位或 64 位时会继续增长，不会像 Java 里定宽度整数那样静默回绕：

README × README ×
f7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-e1f9e52c0bfb4615c10618a113.html
关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计费

2.3 整数：精度、字面量与除法

Python 的 `int` 不区分 `byte`、`short`、`int` 和 `long`。结果超出 32 位或 64 位时会继续增长，不会像 Java 固定宽度整数那样静默回绕：

```
huge_number = 10**30
```

大数字可以用下划线分组，`1_000_000` 与 `1000000` 完全相等。二进制、八进制和十六进制字面量分别使用 `0b`、`0o` 和 `0x` 前缀；`int("ff", 16)` 可以按指定进制解析文本。

Python 的三种相关运算要分清：

表达式	结果	含义
`7 / 2`	`3.5`	真除法，总是产生浮点结果
`7 // 2`	`3`	向负无穷取整的商
`7 % 2`	`1`	与 `//` 配套的余数
`2 ** 8`	`256`	幂运算

`//` 不能简单理解成 Java 整数除法。Java 的整数除法向零截断，而 Python 向负无穷取整：

```
-7 // 2  # -4
-7 % 2   # 1，满足 a == (a // b) * b + (a % b)
```

这在分页偏移、坐标网格和负数时间差中很容易造成迁移错误，涉及负数时应专门测试边界。

2.4 浮点数：看起来简单，不等于精确

Python `float` 通常对应 Java `double`。许多十进制小数不能被二进制浮点数精确表示，所以：

```
0.1 + 0.2 == 0.3  # False
```

这不是 Python 算错了，而是存储近似值带来的结果。测量值等允许微小误差时，用 `math.isclose` 比较，不要手写 `abs(left - right) < ...` 后到处复制容差：

README
README
7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-e1f9e52c0bfb4615c10618a113.html
求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单

2.4 浮点数：看起来简单，不等于精确

Python `float` 通常对应 Java `double`。许多十进制小数不能被二进制浮点数精确表示，所以：

```
0.1 + 0.2 == 0.3  # False
```

这不是 Python 算错了，而是存储近似值带来的结果。测量值等允许微小误差时，用 `math.isclose` 比较，不要手写 `abs(left - right) < ...` 后到处复制容差：

```
from math import isclose

isclose(0.1 + 0.2, 0.3)  # True
```

金额、税率结算等需要十进制规则的场景，常见选择有两种：

1. 用整数保存最小单位，例如把 `19.90` 元保存为 `1990` 分；
2. 使用标准库 `decimal.Decimal`，并从字符串构造：`Decimal("19.90")`。

不要写 `Decimal(0.1)` 后期待得到精确的十进制 `0.1`，因为传入前的 `float` 已经是近似值。`Decimal` 也不是“永不出错”：除法精度、舍入规则仍要根据业务显式设置。

2.5 布尔值与真假值

Java 的 `if` 条件必须是 `boolean`；Python 会对对象做真假值判断。常见假值包括：

- `None`；
- 数值零，如 `0`、`0.0`；
- 空字符串 `""`；
- 空容器，如 `[]`、`{}`、`set()`。

其余对象通常为真。因此检查“集合里有没有内容”可以直接写 `if items:`，比 `if len(items) > 0:` 更符合 Python 习惯。

`and` 与 `or` 返回操作数

`and` 和 `or` 会短路求值，但不保证返回 `bool`，而是返回参与运算的某个对象：

```
nickname = ""
```

README × README
'wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-e1f9e52c0bfb4615c10618a113.html
相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单

2.5 布尔值与真假值

Java 的 `if` 条件必须是 `boolean`；Python 会对对象做真假值判断。常见假值包括：

- `None`；
- 数值零，如 `0`、`0.0`；
- 空字符串 `""`；
- 空容器，如 `[]`、`{}`、`set()`。

其余对象通常为真。因此检查“集合里有没有内容”可以直接写 `if items:`，比 `if len(items) > 0:` 更符合 Python 习惯。

`and` 与 `or` 返回操作数

`and` 和 `or` 会短路求值，但不保证返回 `bool`，而是返回参与运算的某个对象：

```
nickname = ""
display_name = nickname or "匿名用户"  # "匿名用户"
```

这个写法只适用于所有假值都应当触发默认值的场景。如果 `0` 是合法输入，`value or default` 会错误地替换它：

```
retry_count = 0
wrong = retry_count or 3                              # 3，误伤合法的 0
correct = 3 if retry_count is None else retry_count  # 0
```

2.6 `None`：缺失不等于空或零

`None` 表示“没有提供”或“没有结果”，它不是空字符串、零或 `False`。因为 `None` 是单例，判断它应写 `value is None` 或 `value is not None`，而不是 `value == None`。

当空字符串和缺失值语义不同时，应显式区分：例如用户没有填写昵称可以用 `None`，用户填写后又清空可能是 `""`。究竟是否把两者都显示为“匿名用户”，是业务规则，而不是 Python 自动替你决定的。

2.7 字符串：不可变的 Unicode 文本

README README
lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-e1f9e52c0bfb4615c10618a113.html
... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

2.7 字符串：不可变的 Unicode 文本

单引号和双引号没有语义差异，选择能减少转义的一种。Python 没有单独的 `char` 类型，`"Python"[0]` 得到的仍是长度为 1 的 `str`。

`str` 和 Java `String` 一样不可变：`strip`、`replace`、`lower` 等方法都会返回新字符串，不会修改原对象。

```
raw_name = "  Ada  "
clean_name = raw_name.strip()
print(raw_name)    # 原字符串仍保留空格
print(clean_name)  # "Ada"
```

常用操作可以按意图记忆：

意图	写法	说明
清理两端空白	`text.strip()`	返回新字符串
按分隔符拆分	`text.split(",")`	返回字符串列表
连接多个片段	`"`	".join(parts)`
替换内容	`text.replace("old", "new")`	返回新字符串
插入值	`f"你好，{name}"`	f-string 中可写表达式和格式规则

`str` 表示文本，`bytes` 表示原始字节。读写网络或文件时通常要通过编码在两者之间转换；第 10 章会在文件场景中继续讲解。

2.8 多行字符串与 `dedent`

三单引号或三双引号可以创建跨行字符串，并保留其中的换行和缩进。它适合 docstring 和小段文本模板，但它不是通用的“块注释”。

多行字符串写在缩进代码块内时，代码缩进也会成为内容。标准库 `textwrap.dedent` 可以移除每个非空行共有的前导空白：

```
from textwrap import dedent
```

2.8 多行字符串与 `dedent`

三单引号或三双引号可以创建跨行字符串，并保留其中的换行和缩进。它适合 docstring 和小段文本模板，但它不是通用的“块注释”。

多行字符串写在缩进代码块内时，代码缩进也会成为内容。标准库 `textwrap.dedent` 可以移除每个非空行共有的前导空白：

```
from textwrap import dedent

message = dedent("""\
    第一行没有额外的代码缩进。
    第二行也与第一行左对齐。
""")
```

开头的 `\` 会避免三引号之后立即产生一个多余换行。`dedent` 只整理共同缩进，不会自动删除末尾换行；是否保留末尾换行应由输出或模板需求决定。

2.9 综合场景：用“分”计算订单金额

假设商品单价是 `1990` 分，数量是 `3`。核心计算全程使用整数：

```
unit_price_cents = 1_990
quantity = 3
total_cents = unit_price_cents * quantity
yuan, cents = divmod(total_cents, 100)
total_text = f"{yuan}.{cents:02d}"
```

这里 `divmod(total_cents, 100)` 一次返回整除结果和余数；格式化只负责展示，不让浮点数参与核心金额计算。真实支付系统还要定义负数、币种、小数位数和舍入规则，本例只处理非负的人民币分值。

常见错误速查

错误写法或想法	问题	更合适的做法
`int("3.14")`	文本不是合法整数字面量	先明确舍入规则或拒绝输入

这里 `divmod(total_cents, 100)` 一次返回整除结果和余数；格式化只负责展示，不让浮点数参与核心金额计算。真实支付系统还要定义负数、币种、小数位数和舍入规则，本例只处理非负的人民币分值。

常见错误速查

错误写法或想法	问题	更合适的做法
`int("3.14")`	文本不是合法整数字面量	先明确舍入规则或拒绝输入
`0.1 + 0.2 == 0.3`	浮点数是近似表示	`math.isclose` 或 `Decimal`
`bool("False")` 是 `False`	非空字符串都是真值	显式解析允许的文本
`count or 10`	会把合法的 `0` 当成缺失	显式判断 `count is None`
`value == None`	没表达单例身份语义	`value is None`
认为 `strip()` 会原地修改	`str` 不可变	接住返回的新字符串
把 `//` 当 Java 整数除法	负数时取整方向不同	为负数输入单独测试

Java 心智切换

Python 的类型属于对象，不属于一个不可改变的变量槽位； `int` 省去了固定宽度溢出的静默回绕，但浮点误差、除法语义和真假值仍需明确处理。尤其不要把 Python 的隐式真假值理解成“更宽松的 Java boolean”——它是一套需要主动考虑空、零和缺失值的协议。

本章小结

- `type` 看准确类型， `isinstance` 通常更适合考虑继承关系的判断；
- `int` 可按需增长， `/` 、 `//` 和 Java 整数除法的语义不同；
- `float` 适合近似数值，金额优先使用整数最小单位或 `Decimal` ；
- `and` 、 `or` 返回操作数，默认值写法要警惕合法假值；
- `None` 用 `is` 判断，不能与空字符串或零混为一谈；
- `str` 不可变，多行字符串会保留换行与缩进。

小练习

README README
wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-e1f9e52c0bfb4615c10618a113.html
相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单

`0.1 + 0.2 == 0.3`	浮点数是近似表示	`math.isclose` 或 `Decimal`
`bool("False")` 是 `False`	非空字符串都是真值	显式解析允许的文本
`count or 10`	会把合法的 `0` 当成缺失	显式判断 `count is None`
`value == None`	没表达单例身份语义	`value is None`
认为 `strip()` 会原地修改	`str` 不可变	接住返回的新字符串
把 `//` 当 Java 整数除法	负数时取整方向不同	为负数输入单独测试

Java 心智切换

Python 的类型属于对象，不属于一个不可改变的变量槽位； `int` 省去了固定宽度溢出的静默回绕，但浮点误差、除法语义和真假值仍需明确处理。尤其不要把 Python 的隐式真假值理解成“更宽松的 Java boolean”——它是一套需要主动考虑空、零和缺失值的协议。

本章小结

- `type` 看准确类型， `isinstance` 通常更适合考虑继承关系的判断；
- `int` 可按需增长， `/` 、 `//` 和 Java 整数除法的语义不同；
- `float` 适合近似数值，金额优先使用整数最小单位或 `Decimal` ；
- `and` 、 `or` 返回操作数，默认值写法要警惕合法假值；
- `None` 用 `is` 判断，不能与空字符串或零混为一谈；
- `str` 不可变，多行字符串会保留换行与缩进。

小练习

1. 先猜测再运行： `bool("0")` 、 `bool(0)` 、 `isinstance(False, int)` 分别是什么？解释原因。
2. 输入一个秒数 `-1` 、 `0` 、 `59` 、 `60` 或 `3661` ，使用 `divmod` 拆成时、分、秒；观察负数时的结果，并写下这是否符合你的业务规则。
3. 实现 `parse_switch(text: str) -> bool` ：只接受忽略大小写和两端空白后的 `true/false` ，其他文本抛出 `ValueError` 。不要使用 `bool(text)` 代替解析。
4. 输入商品单价（分）和数量，输出以元为单位、保留两位小数的总价。核心计算不得使用 `float` ；为零元、个位数分和大整数各准备一个测试数据。

```python
"""第 2 章: 数字、布尔值、None、字符串和多行文本。"""

from decimal import Decimal
from math import isclose
from textwrap import dedent


def print_section(title: str) -> None:
    """输出分段标题，让运行结果能与 README 小节对应。"""
    print(f"\n=== {title} ===")


def demonstrate_types_and_conversion() -> None:
    """演示运行时类型、继承关系以及显式转换的边界。"""
    print_section("2.1–2.2 类型与转换")

    samples = (42, 3.14, True, "Python", None)
    for value in samples:
        print(f"{value!r:<10} -> {type(value).__name__}")

    # isinstance 会考虑继承关系，而 bool 是 int 的子类。
    print("isinstance(True, int):", isinstance(True, int))
    print("type(True) is int:", type(True) is int)

    count = int("42")
    ratio = float("0.75")
    label = str(42)
    print("显式转换:", count, ratio, label, type(label).__name__)

    # 教学反例：int 不会猜测你想如何处理小数，错误被限制在本小节内。
    try:
        int("3.14")
    except ValueError as error:
        print("int('3.14') 转换失败:", type(error).__name__)

    # bool 判断字符串是否为空，并不会解析字符串写的是 true 还是 false。
```

```
13  def demonstrate_types_and_conversion() -> None:
35
36      # bool 判断字符串是否为空，并不会解析字符串写的是 true 还是 false。
37      print("bool(''):", bool(""), "bool('False'):", bool("False"))
38
39
40  def demonstrate_integers() -> None:
41      """演示任意精度整数、进制和 Python 的整除规则。"""
42      print_section("2.3 整数与除法")
43
44      huge_number = 10**30
45      readable_million = 1_000_000
46      print("大整数:", huge_number)
47      print("下划线不改变数值:", readable_million == 1000000)
48      print("十六进制 ff:", int("ff", 16), "二进制字面量:", 0b1010)
49
50      print("7 / 2 =", 7 / 2)
51      print("7 // 2 =", 7 // 2, "7 % 2 =", 7 % 2)
52      print("-7 // 2 =", -7 // 2, "-7 % 2 =", -7 % 2)
53
54      quotient: int, remainder: int = divmod(-7, 2)
55      reconstructed: int = quotient * 2 + remainder
56      print("divmod(-7, 2):", (quotient, remainder))
57      print("商和余数可还原被除数:", reconstructed)
58
59
60  def demonstrate_floats() -> None:
61      """让浮点近似误差可见，并展示两种合适的处理方式。"""
62      print_section("2.4 浮点数与精确小数")
63
64      result: float = 0.1 + 0.2
65      print("0.1 + 0.2:", result)
66      print("直接比较 0.3:", result == 0.3)
67      print("isclose 比较:", isclose(result, 0.3))
68
69      # 从字符串构造 Decimal，避免先把二进制浮点近似值带进来。
70      exact_result: Decimal = Decimal("0.1") + Decimal("0.2")
```

```python
def demonstrate_floats() -> None:
    exact_result: Decimal = Decimal("0.1") + Decimal("0.2")
    print("Decimal 精确计算:", exact_result)
    print("Decimal 直接比较 0.3:", exact_result == Decimal("0.3"))


def format_total_yuan(unit_price_cents: int, quantity: int) -> str:
    """使用整数分计算非负订单总价，并返回固定两位小数的元字符串。"""
    if unit_price_cents < 0 or quantity < 0:
        raise ValueError("单价和数量必须是非负整数")

    total_cents: int = unit_price_cents * quantity
    yuan: int, cents: int = divmod(total_cents, 100)
    return f"{yuan}.{cents:02d}"


def split_seconds(total_seconds: int) -> tuple[int, int, int]:
    """用两次 divmod 把秒数拆成小时、分钟和秒。"""
    hours: int, remaining_seconds: int = divmod(total_seconds, 60 * 60)
    minutes: int, seconds: int = divmod(remaining_seconds, 60)
    return hours, minutes, seconds


def parse_switch(text: str) -> bool:
    """只接受忽略大小写与两端空白后的 true/false。"""
    normalized: str = text.strip().lower()
    if normalized == "true":
        return True
    if normalized == "false":
        return False
    raise ValueError(f"开关只能是 true 或 false，实际收到 {text!r}")


def demonstrate_boolean_and_none() -> None:
    """演示真假值、短路默认值与 None 的正确判断。"""
    print_section("2.5–2.6 布尔值与 None")
```

```python

def demonstrate_boolean_and_none() -> None:
    """演示真假值、短路默认值与 None 的正确判断。"""
    print_section("2.5–2.6 布尔值与 None")

    values: tuple[None, Literal[0], float, Literal...] = (None, 0, 0.0, "", [], "0", [0])
    for value in values:
        print(f"bool({value!r}) = {bool(value)}")

    nickname: Literal[''] = ""
    display_name: Literal['匿名用户'] = nickname or "匿名用户"
    print("空昵称使用默认值:", display_name)

    # 教学反例：0 是合法的重试次数，却会被 or 当作假值替换。
    retry_count = 0
    wrong_retry_count: Literal[3] = retry_count or 3
    correct_retry_count: Literal[3] | Literal[0] = 3 if retry_count is None else retry_count
    print("or 误伤合法的 0:", wrong_retry_count)
    print("显式区分 None 后:", correct_retry_count)

    missing = None
    print("缺失值使用 is None:", missing is None)


def demonstrate_strings() -> None:
    """演示字符串不可变性以及拆分、连接和替换。"""
    print_section("2.7 字符串")

    raw_language = "  Python  "
    language: LiteralString = raw_language.strip()
    print("原字符串保留空格:", repr(raw_language))
    print("strip 返回新字符串:", repr(language))
    print("首字符仍是 str:", language[0], type(language[0]).__name__)

    lowered: LiteralString = language.lower()
    replaced: LiteralString = language.replace("Python", "Python 3.14")
    print(f"{language} / lower={lowered} / length={len(language)}")
```

```
125 def demonstrate_strings() -> None:
136     replaced: LiteralString = language.replace("Python", "Python 3.14")
137     print(f"{language} / lower={lowered} / length={len(language)}")
138     print("replace 后:", replaced, "; 原值:", language)
139
140     parts: list[LiteralString] = "a,b,c".split(",")
141     print("切分结果:", parts)
142     print("连接结果:", " | ".join(parts))
143
144
145 def demonstrate_multiline_text() -> None:
146     """演示三引号字符串如何保留换行，以及 dedent 的作用。"""
147     print_section("2.8 多行字符串")
148
149     message: str = dedent("""\
150         第一行：三引号字符串可以跨行。
151         第二行：dedent 可以移除代码的共同缩进。
152     """)
153     print(message, end="")
154     print("末尾是否保留换行:", message.endswith("\n"))
155
156
157 def demonstrate_order_total() -> None:
158     """组合整数、divmod 和字符串格式化完成一个小型业务例子。"""
159     print_section("2.9 综合示例：订单金额")
160
161     for unit_price_cents, quantity in ((1_990, 3), (5, 1), (0, 8)):
162         total_yuan: str = format_total_yuan(unit_price_cents, quantity)
163         print(f"{unit_price_cents} 分 × {quantity} = {total_yuan} 元")
164
165
166 def demonstrate_readme_exercises() -> None:
167     """运行 README 末尾的四组练习与边界输入。"""
168     print_section("README 小练习")
169
170     print("练习 1：真值与 bool/int 继承关系")
171     print("bool('0'):", bool("0"))
```

```
def demonstrate_readme_exercises() -> None:
    print("bool('0'):", bool("0"))
    print("bool(0):", bool(0))
    print("isinstance(False, int):", isinstance(False, int))

    print("练习 2: 使用 divmod 拆分秒数")
    for total_seconds in (-1, 0, 59, 60, 3661):
        print(f"  {total_seconds:>4} 秒 -> {split_seconds(total_seconds)}")
    print(
        "  -1 秒得到前一天的 23:59:59; "
        "若业务不接受负数，应在入口主动拒绝"
    )

    print("练习 3: 显式解析开关")
    for switch_text in (" true ", "FALSE"):
        print(f"  parse_switch({switch_text!r}) -> {parse_switch(switch_text)}")
    try:
        parse_switch("yes")
    except ValueError as error:
        print("  非法开关:", error)

    print("练习 4: 只用整数分计算金额")
    order_cases = ((0, 10), (7, 1), (123_456_789_012_345, 3))
    for unit_price_cents, quantity in order_cases:
        result: str = format_total_yuan(unit_price_cents, quantity)
        print(f"  {unit_price_cents} 分 × {quantity} = {result} 元")


def main() -> None:
    demonstrate_types_and_conversion()
    demonstrate_integers()
    demonstrate_floats()
    demonstrate_boolean_and_none()
    demonstrate_strings()
    demonstrate_multiline_text()
    demonstrate_order_total()
    demonstrate_readme_exercises()
```

```
 6  def demonstrate_readme_exercises() -> None:
 1      )
 2
 3      print("练习 3: 显式解析开关")
 4      for switch_text in (" true ", "FALSE"):
 5          print(f"  parse_switch({switch_text!r}) -> {parse_switch(switch_text)}")
 6      try:
 7          parse_switch("yes")
 8      except ValueError as error:
 9          print("  非法开关:", error)
10
11      print("练习 4: 只用整数分计算金额")
12      order_cases = ((0, 10), (7, 1), (123_456_789_012_345, 3))
13      for unit_price_cents, quantity in order_cases:
14          result: str = format_total_yuan(unit_price_cents, quantity)
15          print(f"  {unit_price_cents} 分 × {quantity} = {result} 元")
16
17
18  def main() -> None:
19      demonstrate_types_and_conversion()
20      demonstrate_integers()
21      demonstrate_floats()
22      demonstrate_boolean_and_none()
23      demonstrate_strings()
24      demonstrate_multiline_text()
25      demonstrate_order_total()
26      demonstrate_readme_exercises()
27
28
29  if __name__ == "__main__":
30      main()
31
```